



Capstone Project: Multi-Agent Document Intelligence System

Week 4 - Two days to build something real

What We Are Building

A system that reads any document and answers questions about it , with sources

You give it a PDF. Legal contract, medical report, research paper, anything. It reads the full document, breaks it into searchable pieces, stores them, and when you ask a question it finds the right pieces and answers using only what's in the document , never making things up.

Today

Ingestion pipeline plus RAG question answering

Tomorrow

LangGraph agent on top plus a FastAPI endpoint to serve it



Why This Project Specifically

Four weeks of concepts , one project that connects all of them

1

Week 1

Prompt engineering and LLM calls. Used in the RAG answer generation step.

2

Week 2

Chunking, embeddings, vector stores, RAGAS evaluation. That is the entire ingestion pipeline.

3

Week 3

LangGraph agents. RAG system in an agent that can plan and reason.

4

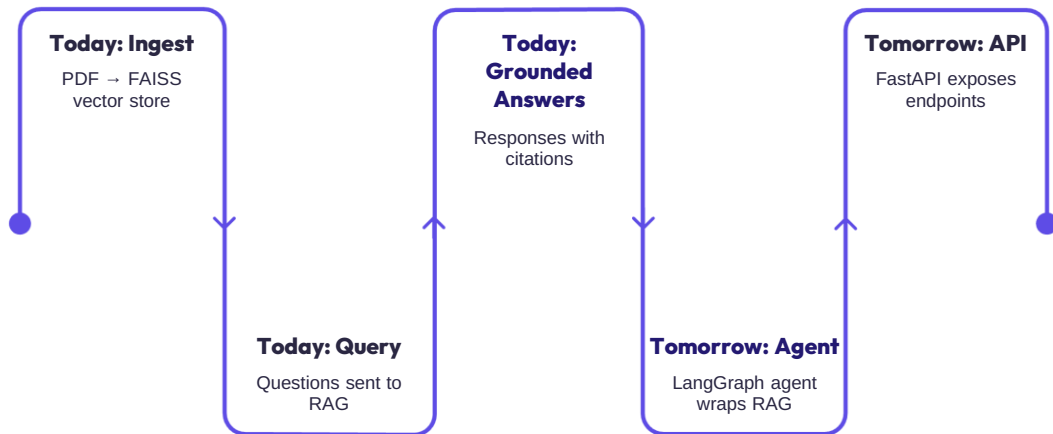
Deployment

FastAPI serves the final system as a real endpoint.

 This is not a new project. It is everything you already built, assembled into one thing that does something genuinely useful.

The Full Architecture

How the two days connect



Today, Foundation

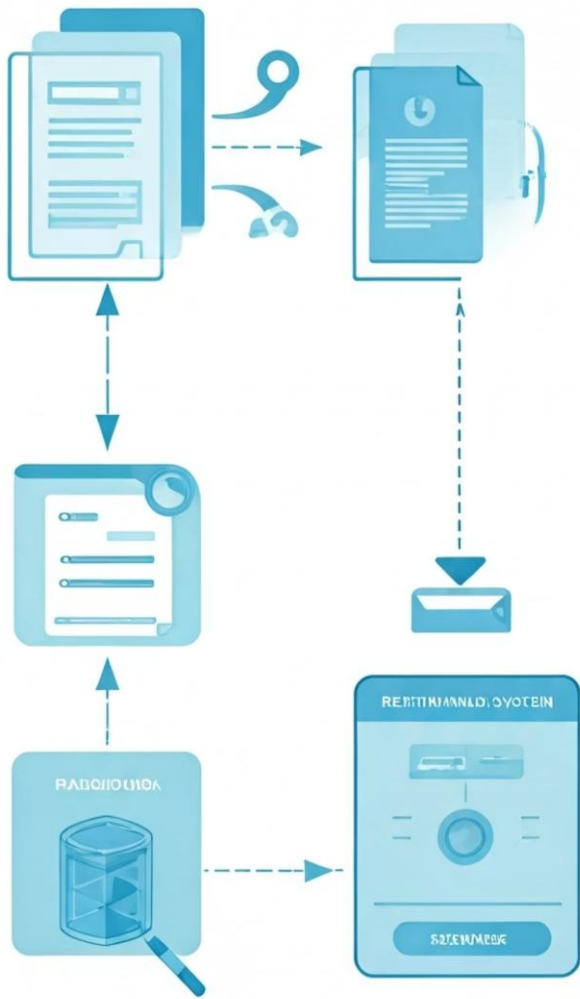
PDF goes in, FAISS vector store comes out, questions go in, grounded answers with citations come out.

Tomorrow, Intelligence Layer

A LangGraph agent that decides when to call the RAG system as a tool, handles multi-step questions, and a FastAPI layer that exposes everything as an API endpoint.

Key Design Decision

Ingestion and retrieval are intentionally separated. You ingest once, query many times. This is how real document systems work.



FAISS index

Today's Focus:

Ingestion and RAG: the two halves of today

First Half , ingestion.py

Load a PDF, chunk it, embed every chunk, save to a local FAISS index.
Run this once per document.

Second Half , rag.py

Load that index, take a user question, retrieve the 4 most relevant chunks, send them to the LLM with strict grounding instructions, return the answer plus which pages it came from.

These two scripts are independent by design. Ingestion runs offline. RAG runs on demand. Tomorrow FastAPI will sit in front of both.

Step 1, Document Ingestion

From raw PDF to a searchable vector store



PyPDFLoader

Reads the PDF page by page and gives us raw text with page number metadata attached.



HuggingFaceEmbeddings

Converts every chunk into a vector using all-MiniLM-L6-v2, a small and fast model that runs locally with no API key.



RecursiveCharacterTextSplitter

Breaks text into 500-character chunks with 50-character overlap. The overlap exists so a sentence that lands at a boundary does not lose its context.

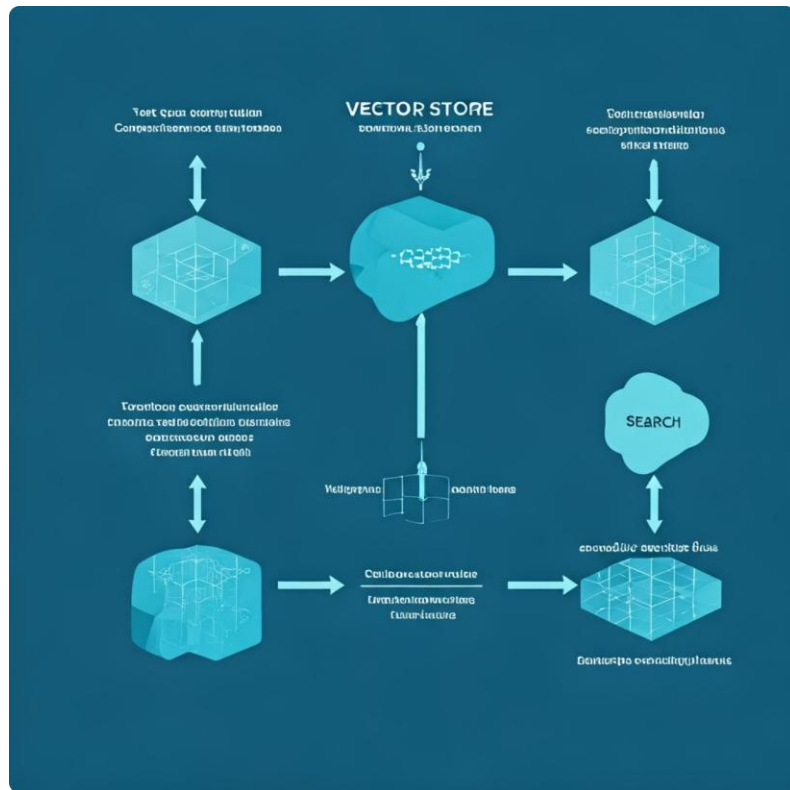


FAISS Index

Stores all those vectors on disk at ./faiss_index. From this point the PDF is never needed again.

What a Vector Store Actually Does

One idea worth understanding properly before you move on



Semantic Search, Not Keyword Search

When you ask a question, the system does not search for matching keywords. It converts your question into a vector using the same embedding model, then finds the chunks whose vectors are closest in meaning.

This is why **"remote work rules"** can find a chunk that says **"employees working from home"**. No shared words, same meaning. That is the difference between keyword search and semantic search.

What FAISS Is

FAISS is just the library that makes the nearest-neighbor lookup fast. It stands for **Facebook AI Similarity Search**.

Step 2 , RAG with Citations

Retrieval, grounding, and why citations are non-negotiable



Retrieve 4 Chunks

Format Context Block

LLaMA 3.3-70B Answer

The prompt explicitly says: answer using **ONLY** the context below, and state which section or page you drew from. Without that instruction, the model will blend in its own training data , which may be wrong, outdated, or simply not from this document.

The system returns two things: the answer, and a sources list showing which pages each retrieved chunk came from. This is the same faithfulness principle from RAGAS week, applied at generation time.

 Without strict grounding instructions, the model will blend in its own training data , which may be wrong, outdated, or simply not from this document.

The Two Files We Built

ingestion.py and **rag.py** , what each one owns

ingestion.py

Runs once from the command line: `python ingestion.py --file report.pdf`. It loads, chunks, embeds, and saves. One status line when done. It does not know anything about questions or answers.

rag.py

Exposes a single function: `get_answer(question)` returns a dict with `answer` and `sources`. It loads the FAISS index, retrieves, generates, and returns. It does not know anything about how the question arrived , command line, FastAPI, or a LangGraph agent can all call it the same way.

❏ This separation is intentional. Each file has one job and does not care how the other works.

Why We Separated Ingestion from Retrieval

A design decision worth understanding, not just copying

Ingestion is Slow

Embedding thousands of chunks takes time and CPU. You only want to do it once when a document changes.

Retrieval Must Be Fast

A user asking a question should get an answer in seconds, not wait for re-embedding.

Combined = Unusable

If you combined them into one script that embeds the document every time someone asks a question, the system would be unusable at any real scale.

The pattern: **ingest offline, serve online**. This is how every production document system works, from simple RAG to enterprise search.

